inherited-f4-batched: fewer counted rewrites (2026-11-12)

`inherited-f4-batched`: fewer counted rewrites, same decisions (2026-11-12)

Owner's ask (2026-11-12): "do the rest of this stuff. Keep working on these things". This is the third lever named after the previous round: the eager rewrite of kept rows was 11 to 20 % of `inherited-f4`'s instructions (`../inherited_f4_next_20261112/` profile).

What changed

Written by an Opus subagent to a frozen brief, reviewed in the main session; `src/cryptanalysis/inherited_f4.rs`, `src/cryptanalysis/koblitz_groebner.rs`, engine registration in `src/cryptanalysis/ic_framework/solvers.rs`.

Between two splits a search node specialises several times (the split variable, then each forced variable), and the default eager mode rewrites every kept row of the reduced basis at every specialisation, although a node that is refuted or ends at a leaf never reads most of them. With the switch `koblitz_groebner::with_batched_rewrite` on (engine `inherited-f4-batched`):

- kept rows stay in their old column layout at a specialisation; displaced rows are rewritten and re-reduced exactly as before;
- a row is brought current when it is read (pivot of a re-reduction, tail row, completion, closure), through the composition of every step since its last write, with the eager kernel and the eager per-row charge;
- at a split every stale row is brought current once, before the two children share the basis (charged inside the counted region).

Why decisions cannot change. A rewrite changes only the column layout of a row, not the polynomial; the composed map is the composition of the step maps, so a row brought current late is the row the eager path holds. Kept / displaced is decided by pivot columns, tracked eagerly in both modes, and displaced rows are inserted in the same order against the same pivots. The basis after every operation is therefore identical, and so are the linear tail, refutations, forced variables, splits and roots. Every tree statistic (reductions, splits, propagations, infeasible branches, eliminated, oversize, tail enumerations, degree reached, exhausted) is identical; only `ops` (word reads and writes of the specialisation) changes.

Why the count only drops for work no longer done. A batched rewrite across steps e→now charges the first read and the last write of that chain, never more than the eager steps it replaces; the split materialisation sits where the eager parent's last rewrite sat; rows never read before a leaf or a refutation are not rewritten at all. Tests check the stronger statement that `ops − rewrite words` is identical in both modes on every system. Worked example (pin system 23, 10 unknowns): 80 → 66 word operations, the 14 being 18 words of eager kept-row rewrites replaced by 4 words of two late single-row rewrites; seven of the nine eagerly rewritten rows were never read.

Tests:
`koblitz_groebner::tests::batched_rewrite_leaves_decisions_and_never_adds_word_xors` (33 pin systems + 8 dense at 12/14/16/20 unknowns, both modes, cold and warm),
`inherited_f4::tests::batched_rewrite_builds_identical_bases_with_no_more_work` (80 basis chains, both kernels), and
`ic_framework::solvers::tests::size_rule_and_inherited_fold_match_the_reference` (extended: batched gives the same solutions with no more ops). Ops ratio batched/eager on the 41 test systems: 0.9843 in total, 1.0 worst, 21 systems cheaper `[pcc:unverified]`.

Known cost before measuring, and the fix

The first version (1da7694876, merged in #4005) was checked unpaired by the subagent on Koblitz 19:10:2 and 23:12:2: 0.879 and 0.830 of the ops per target and **8 to 13 % more wall time** `[pcc:unverified]`. Callgrind put the cost in the allocator: one allocation and free per row brought current on its own (about 27 M allocator instructions against 9 M for eager on 23:12:2, while the batched arm already executed about 3 % fewer instructions overall).

4f0eca9eb4 rewrites such rows into spans of a per-thread write-once arena (zeroed 16K-word chunks; rows above 4K words keep their own buffer) and reads every row through a pointer and length with an owner that keeps the buffer alive. No counted value moves: the test `koblitz_groebner::tests::batched_rewrite_counts_are_pinned` pins `ops`, `bitop/v1` word ops and hash probes of the 41 systems, cold and warm, to the values of 1da7694876. Callgrind after the fix, `HybridF4::solve` on 23:12:2: eager 273.2 M, batched 249.5 M instructions (0.913); the subagent's unpaired wall check: batched 0.92 to 0.96 of eager at 23:12:2 and parity within noise at 19:10:2 `[pcc:unverified]`. The overhead outside the counted unit no longer grows, so a lower S here rests on counted work that disappears without a hidden wall cost.

Run 1: paired per call (`run_ab.sh`)

`ic descent --solver inherited-f4 --solver inherited-f4-batched --solver fes-f2`, the 13 cells of the previous study (14 to 32 unknowns), 6 targets × 3 repeats, one thread, frozen binary (`IC_BIN`).

Measured at 4f0eca9eb4 (the arena fix), frozen binary. Expectation: ops batched/old ≤ 1 on every target, about 0.83 to 0.88 at 20 to 24 unknowns, closer to 1 below; 0 disagreements; wall batched/old at or below 1 (about 0.9 to 1.0) from 20 unknowns up, near 1 below. Falsifier: any target with more ops, any disagreement, no ops reduction at 20 unknowns and up, or a median paired wall ratio above 1.05 at 20 unknowns and up.

Run 2: end to end (`run_e2e.sh`)

`ic bench --sweep` on the six e2e-ladder instances, `inherited-f4` and `inherited-f4-batched`, 3 repeats. Expectation: same planted logarithms recovered, S of the batched row below `inherited-f4`'s roughly by the ops ratio of its cell (the solver is 88 to 100 % of the row's GAE): about 0.85 to 0.9 at Koblitz 19 and 23, closer to 1 at the small instances. Falsifier: batched S above `inherited-f4`'s on any instance, or not below at Koblitz 19 and 23.

Results `[pcc:unverified]`

Both runs used the binary built at 4f0eca9eb4 from a clean tree (SHA-256 in `results/ic-sha256.txt`, `results_e2e/ic-sha256.txt`), one thread, local container, not isolated.

Run 1: paired per call

Record `benchmarks/records/20261112T1829Z-inherited-f4-batched-ab.json` (measured). Full table `RESULTS_AB.md`, graph `results_ab.svg/png`.

unknowns	word XORs batched / eager, K / R	wall batched / eager, K / R
14	0.851 / 0.855	1.13 / 1.08
16	0.844 / 0.846	1.11 / 1.14
18	0.810 / 0.808	0.99 / 0.99
20	0.878 / 0.883	1.04 / 1.08
22	— / 0.840	— / 1.02
24	0.830 / 0.828	0.91 / 0.95
30 (K)	0.760	0.90
32 (K)	0.715	0.85

Median over targets of the per-target ratio, 6 targets × 3 repeats.

- **The counted work drops on every target**: 0.70 to 0.91 per target, never more, median 0.84 over the cells, more than expected below 20 unknowns (0.81 to 0.86 instead of "closer to 1"). 0 disagreements with `fes-f2`; every target decided by every engine.
- **Wall time follows from 24 unknowns up** (0.85 to 0.95, batched below eager on 22 of 24 targets there), **but not below**: 1.08 to 1.14 at 14 to 16 unknowns, about 1.0 at 18, 1.04 and 1.08 at 20, 1.02 at 22. The pre-stated falsifier (a median paired wall ratio above 1.05 at 20 unknowns and up) **fired on random 19:10:2 (1.08)**. The arena fix removed the allocation cost but not all of the per-row overhead (the composed step-plan lookup per row, which the subagent left in place): on small systems the rows brought current one at a time still cost more than the bulk rewrite they replace.

Run 3: paired per call after the plan-builder change (protocol, stated before the run)

cd4dcfb3f1 (branch-light plan builder, cached own-step plan, short-list epoch count, pooled arena chunks; `inherited_f4.rs` only, counts pinned by `batched_rewrite_counts_are_pinned`). Same launcher and cells as run 1, frozen binary, results in `results_run3/`, own record.

Expectation: per-target `ops` of both engines identical to run 1 on every target (same seeds, counts pinned); 0 disagreements; wall batched/eager about 1.06 at 14 unknowns, 1.12 at 16, at or below 1.0 from 20 up (the subagent's unpaired check: 1.061, 1.126, 1.001, 0.934). Falsifier: any per-target `ops` differing from run 1, any disagreement, or a median paired wall ratio above 1.05 at 20 unknowns and up.

Run 3 result `[pcc:unverified]` (record `benchmarks/records/20261112T1938Z-inherited-f4-batched-ab-run3.json`, `RESULTS_AB_RUN3.md`, binary built at cd4dcfb3f1 from a clean tree):

- **Counts unchanged**: all 234 target × engine pairs have exactly the per-target `ops` of run 1; 0 disagreements with `fes-f2`.
- **Wall batched / eager, run 1 → run 3**: K 13 1.13 → 1.09, R 13 1.08 → 1.04, K 15 1.11 → 1.11, R 15 1.14 → 1.12, K 17 0.99 → 1.05, R 17 0.99 → 1.01, K 19 1.04 → 0.98, R 19 1.08 → 1.10, R 21 1.02 → 0.95, K 23 0.91 → 0.97, R 23 0.95 → 0.89, K 29 0.90 → 0.89, K 31 0.85 → 0.85. The changes go both ways by up to 0.07, the size of the run-to-run spread; the plan-builder change does not measurably move the paired wall. **The pre-stated falsifier fired again on random 19:10:2 (1.10)**, where the subagent's unpaired check had suggested parity (1.001).
- Conclusion for this lever: below 20 unknowns the batched mode stays 4 to 12 % above eager on wall; the remaining cost is the composed-plan builds, which are counted work and cannot be skipped without changing the count. From 22 unknowns up it is below eager (0.85 to 0.97). The counted gain (0.70 to 0.91 of the word XORs) holds at every size.

Run 2: end to end

Record `benchmarks/records/20261112T1831Z-inherited-f4-batched-e2e.json` (measured). Full table `RESULTS_E2E.md`, graph `results_e2e.svg/png`. Every configuration recovered its planted logarithm in every repeat, and so did every matched rho run.

instance (unknowns)	rho S	pair table S	`inherited-f4` S	`inherited-f4-batched` S	batched / eager	ms per call eager / batched
Koblitz 13 (14)	2.89	197.8	137.9	120.2	0.871	0.83 / 0.93
random 15 (16)	2.28	473.7	262.7	223.9	0.852	1.38 / 1.52
Koblitz 17 (18)	1.73	493.2	304.5	249.7	0.820	2.78 / 2.76

instance (unknowns)	rho S	pair table S	inherited-f4 S	inherited-f4-batched S	batched / eager	ms per call eager / batched
Koblitz 19 (20)	1.35	1,063.3	465.5	413.6	0.889	4.28 / 4.45
random 21 (22)	1.24	1,975.9	4,299.2	3,631.1	0.845	7.68 / 7.80
Koblitz 23 (24)	0.88	3,986.3	2,051.2	1,674.3	0.816	13.81 / 12.55

Mean over 3 repeats; the repeat ranges of the two engines are listed in `RESULTS_E2E.md` (they overlap, because the repeats differ by target set far more than the engines differ; on each repeat the batched row is below the eager one).

- **The batched row has the lower S on every instance, 0.82 to 0.89×**, as its word-XOR count drops (the solver is 88 to 100 % of the row's GAE). The expectation held on Koblitz 19 and 23 (0.889, 0.816) and was exceeded on the smaller instances.
- **At degree 13 the size rule stays the best row**: `fes-then-inherited` runs `fes-f2` there (S 101.9 in `../size_rule_fold_20261112/`), below the batched row's 120.2. Above 14 unknowns the batched row is now the best algebraic row; the pair table stays best on random 21.
- **What the S drop rests on**: counted work that genuinely disappears (rows never rewritten), at every size. From 24 unknowns up the wall agrees (0.91 per call in the pipeline at Koblitz 23). At 20 unknowns and below, per-call wall in the pipeline is 0.99 to 1.12× of eager: there the drop in S is real counted work, offset by an overhead the counted unit does not see, and should be read with that caveat until the per-row overhead is removed.

No leaderboard row: no replay certificate or validator report (validation gate).

Next

- Remove the remaining per-row overhead (composed step-plan lookup per row, `HashMap` + `RefCell`) so the wall agrees with the count below 24 unknowns.
- Combine the two gains: a size rule with `fes-f2` up to 14 unknowns and `inherited-f4-batched` above, or make the batched rewrite the default once the small-system wall cost is gone (bump `INHERITED_F4_ACCOUNTING_VERSION` then, since the default count changes).

Sources checked

KB (`research-kb`) unavailable in this session.
`src/cryptanalysis/inherited_f4.rs`, `src/cryptanalysis/koblitz_groebner.rs`, `src/cryptanalysis/ic_framework/solvers.rs`, the subagent's Callgrind outputs (scratchpad `cgb/`), `research/size_rule_fold_20261112/README.md`.

Median over targets of the per-target median wall (ms per call, one thread). Paired: median (min–max) over targets of the per-target ratio. `ops` = median per-target word-XOR count (the engines' counted unit). `dec` = targets decided / targets; every decided answer is checked against the reference `fes-f2`.

family	n	vars	inherited-f4 ms	batched ms	wall batched/old paired	targets batched < old	ops old	
K	13	14	0.87	0.92	1.127 (0.84–1.46)	1/6	49,188	
R	13	14	0.86	0.91	1.077 (1.01–1.17)	0/6	52,356	
K	15	16	1.94	2.18	1.111 (0.94–1.20)	2/6	124,202	
R	15	16	1.46	1.66	1.139 (0.92–1.25)	1/6	102,284	
K	17	18	3.49	3.22	0.986 (0.67–1.07)	4/6	250,570	
R	17	18	2.35	2.35	0.993 (0.88–1.11)	4/6	240,186	
K	19	20	3.89	4.16	1.037 (0.89–1.38)	2/6	527,527	
R	19	20	4.69	5.04	1.076 (1.00–1.19)	0/6	561,969	
R	21	22	6.86	7.06	1.020 (0.97–1.14)	2/6	1,056,918	
K	23	24	14.38	13.11	0.913 (0.76–0.97)	6/6	2,503,847	
R	23	24	13.38	12.82	0.949 (0.87–1.03)	4/6	2,306,914	
K	29	30	84.46	75.68	0.896 (0.82–0.95)	6/6	20,022,108	1
K	31	32	190	158	0.853 (0.72–0.94)	6/6	42,855,453	3

Per instance: the matched counted rho (mean S over its runs) and every configuration run end to end. `S` = total group-addition equivalents / sqrt(r), mean over repeats (min–max); `S / rho` = mean s_over_rho; `ok` = every repeat recovered the planted logarithm; `hit` = decomposition hit rate; `tried` = targets tried; `solver` = engine, priced_by, mean calls, budget hits, share of the row's GAE, wall per call.

instance (r)	configuration	S mean (min–max)	S / rho
`icv1-f2m15-tm249-9e24ae34` (r = 5503)	**matched rho** `negation-map r-adding walk: look-ahead, short-cycle escape by doubling, distinguished points, stride starts`, 16 runs	2.28	1.00
	`binary-subspace[dimension=8] + mitm[m=2] + – + walk`	473.71 (473.13–474.62)	207.78
	`binary-subspace[dimension=8] + descent-algebraic[m=2] + inherited-f4 + walk`	262.68 (188.85–365.78)	115.22
	`binary-subspace[dimension=8] + descent-algebraic[m=2] + inherited-f4-batched + walk`	223.90 (162.23–311.58)	98.21
`icv1-f2m21-tm1325-b6925563` (r = 1049239)	**matched rho** `negation-map r-adding walk: look-ahead, short-cycle escape by doubling, distinguished points, stride starts`, 16 runs	1.24	1.00
	`binary-subspace[dimension=11] + mitm[m=2] + – + walk`	1975.87 (1975.84–1975.93)	1597.50
	`binary-subspace[dimension=11] + descent-algebraic[m=2] + inherited-f4 + walk`	4299.15 (4130.60–4530.25)	3475.88

instance (r)	configuration	S mean (min–max)	S / rho
	`binary-subspace[dimension=11] + descent-algebraic[m=2] + inherited-f4-batched + walk`	3631.12 (3490.02–3825.55)	2935.77
`icv1-f2m13-t181-515ee569` (r = 2003)	**matched rho** `negation-map r-adding walk: look-ahead, short-cycle escape by doubling, distinguished points, stride starts`, 16 runs	2.89	1.00
	`binary-subspace[dimension=7] + mitm[m=2] + – + walk`	197.78 (197.66–198.01)	68.38
	`binary-subspace[dimension=7] + descent-algebraic[m=2] + inherited-f4 + walk`	137.95 (117.79–167.62)	47.70
	`binary-subspace[dimension=7] + descent-algebraic[m=2] + inherited-f4-batched + walk`	120.21 (102.87–145.90)	41.57
`icv1-f2m17-tm101-00378d4e` (r = 65587)	**matched rho** `negation-map r-adding walk: look-ahead, short-cycle escape by doubling, distinguished points, stride starts`, 16 runs	1.73	1.00
	`binary-subspace[dimension=9] + mitm[m=2] + – + walk`	493.17 (492.55–493.54)	285.87
	`binary-subspace[dimension=9] + descent-algebraic[m=2] + inherited-f4 + walk`	304.50 (102.55–415.33)	176.51
	`binary-subspace[dimension=9] + descent-algebraic[m=2] + inherited-f4-batched +`	249.66 (84.46–	144.72

instance (r)	configuration	S mean (min–max)	S / rho
	`walk`	340.52)	
`icv1-f2m19-tm797-9c54981b` (r = 262543)	**matched rho** `negation-map r-adding walk: look-ahead, short-cycle escape by doubling, distinguished points, stride starts`, 16 runs	1.35	1.00
	`binary-subspace[dimension=10] + mitm[m=2] + – + walk`	1063.26 (1062.78–1063.52)	785.16
	`binary-subspace[dimension=10] + descent-algebraic[m=2] + inherited-f4 + walk`	465.45 (227.52–594.50)	343.71
	`binary-subspace[dimension=10] + descent-algebraic[m=2] + inherited-f4-batched + walk`	413.64 (202.26–527.86)	305.45
`icv1-f2m23-tm5197-1f85e9e1` (r = 4196903)	**matched rho** `signed-Frobenius r-adding walk, distinguished points (koblitz_signed_frobenius_rho_reference)`, 16 runs	0.88	1.00
	`binary-subspace[dimension=12] + mitm[m=2] + – + walk`	3986.33 (3986.00–3986.59)	4542.18
	`binary-subspace[dimension=12] + descent-algebraic[m=2] + inherited-f4 + walk`	2051.17 (1498.17–2448.60)	2337.18
	`binary-subspace[dimension=12] + descent-algebraic[m=2] + inherited-f4-batched + walk`	1674.33 (1223.11–1998.99)	1907.79

instance (r)	configuration	S mean (min–max)	S / rho